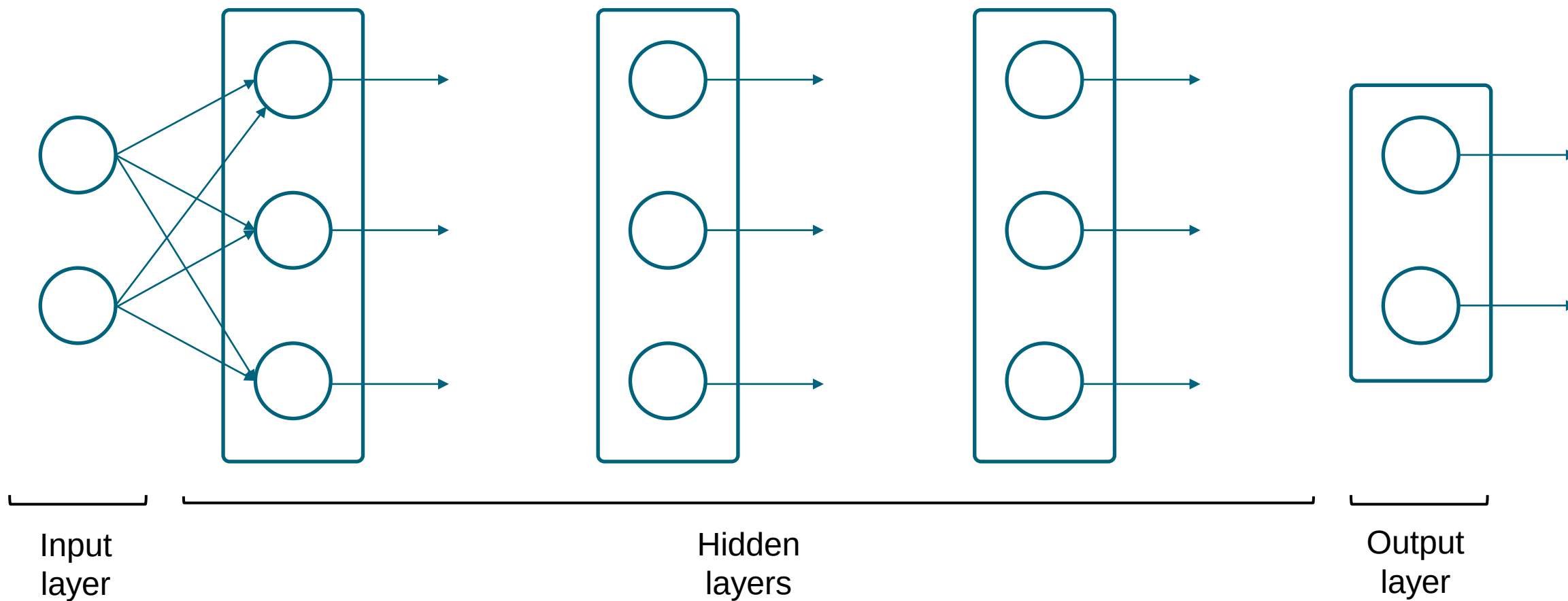
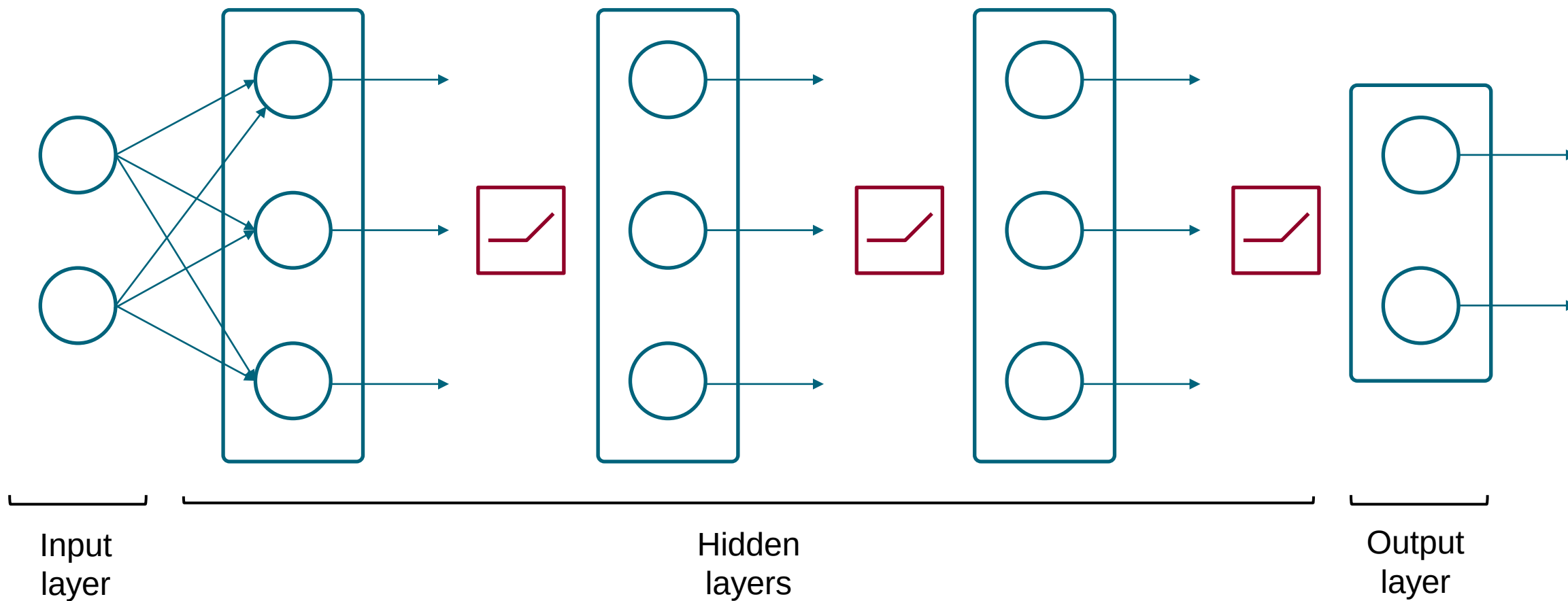


The Role of Activation Functions

Sequential Model



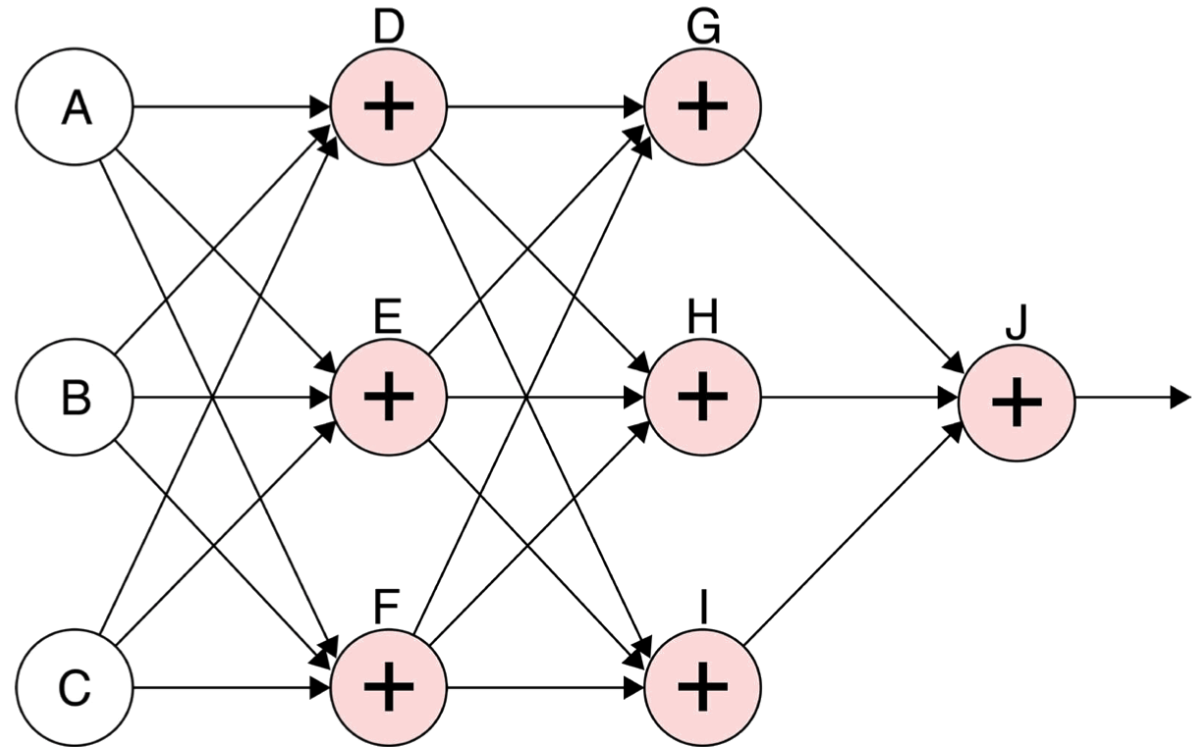
Sequential Model



 Nonlinear function

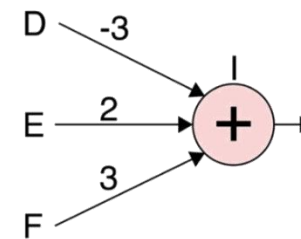
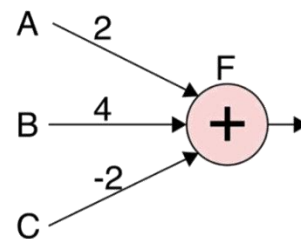
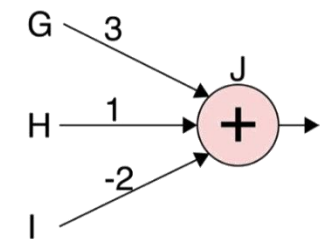
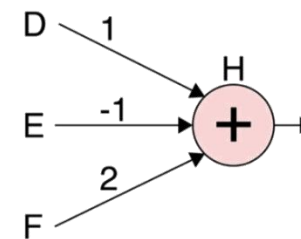
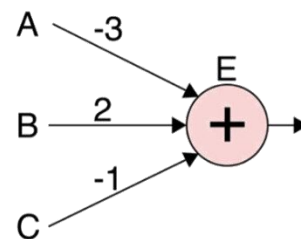
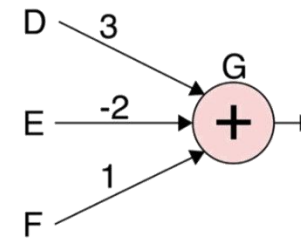
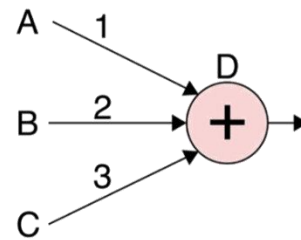
Activation Functions: Why They Matter, Part I

- Fully connected network without activation functions



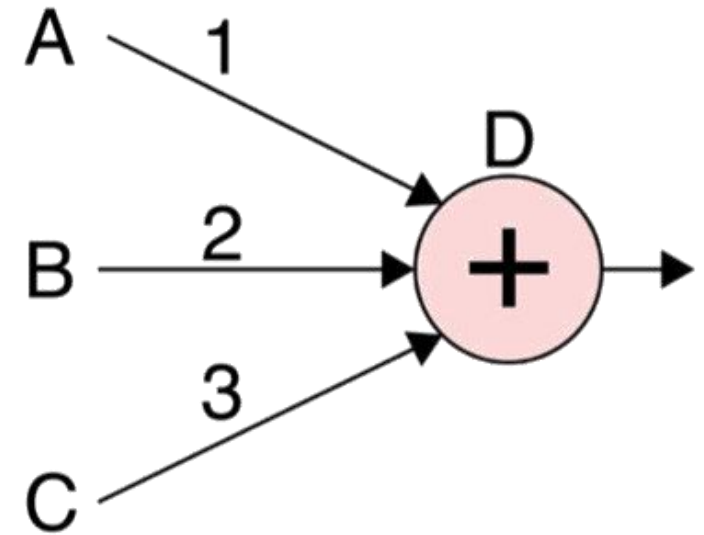
Activation Functions: Why They Matter, Part II

- Fully connected network without activation functions
- Every unit from D to J computes some combination of the inputs A, B, and C



Activation Functions: Why They Matter, Part III

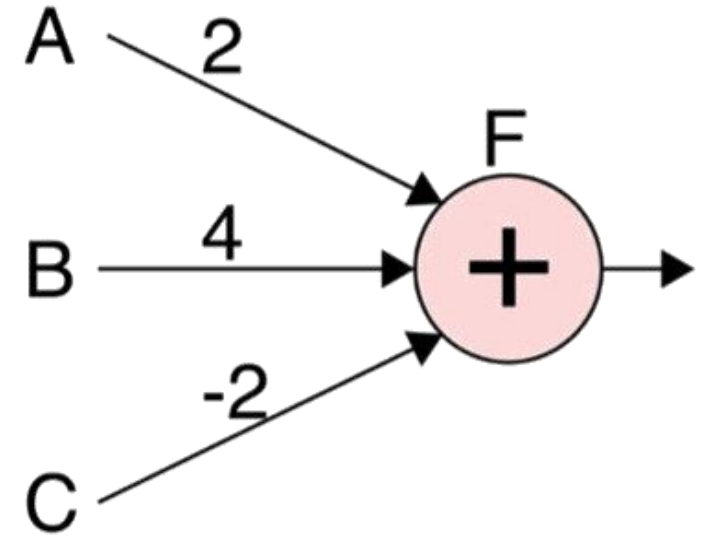
- Fully connected network without activation functions
- Every unit from D to J computes some combination of the inputs A, B, and C



$$D = A + 2B + 3C$$

Activation Functions: Why They Matter, Part IV

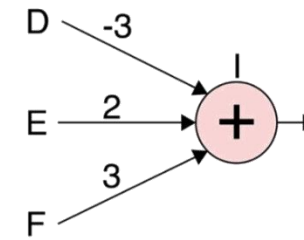
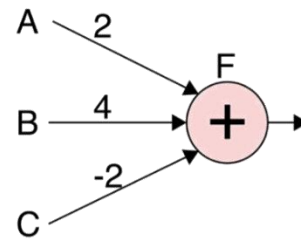
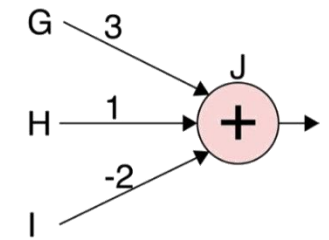
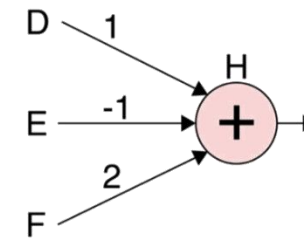
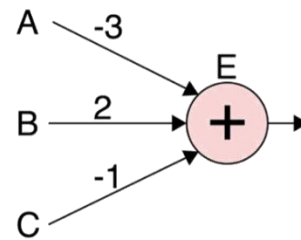
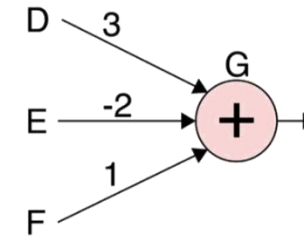
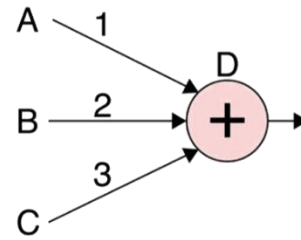
- Fully connected network without activation functions
- Every unit from D to J computes some combination of the inputs A, B, and C



$$F = 2A + 4B - 2C$$

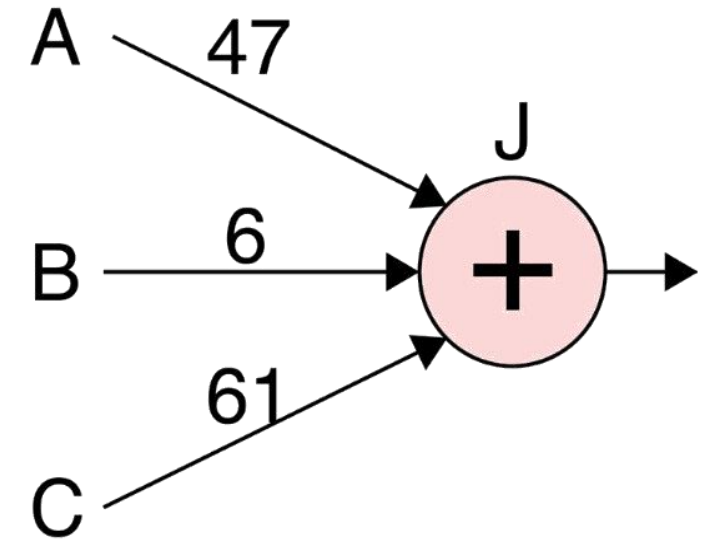
Activation Functions: Why They Matter, Part V

- Fully connected network without activation functions
- Every unit from D to J computes some combination of the inputs A, B, and C



Activation Functions: Why They Matter, Part VI

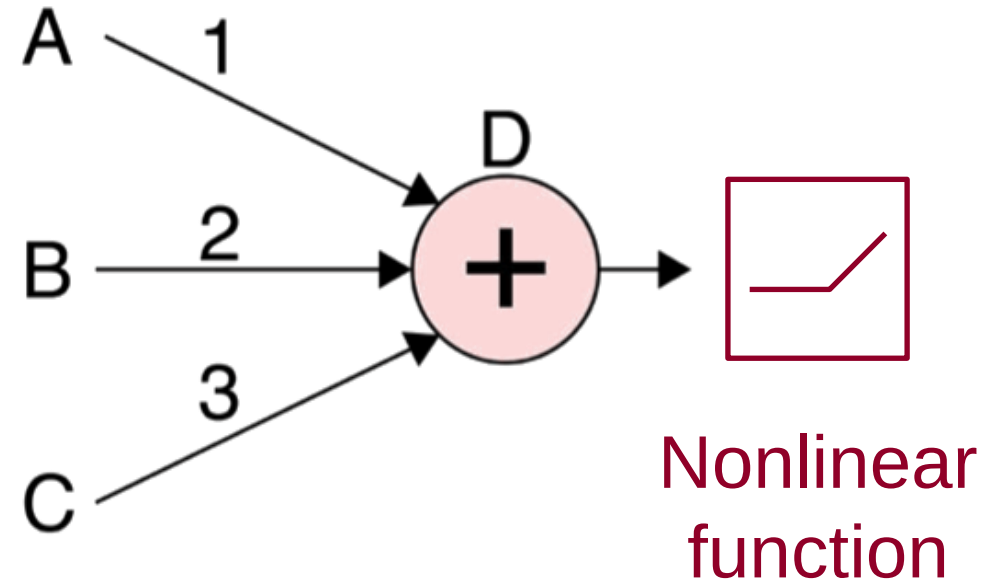
- Fully connected network without activation functions
- Every unit from D to J computes some combination of the inputs A, B, and C
- **Network collapse:** the output of J is just this combination of A, B, and C



$$J = 47A + 6B + 61C$$

The Purpose of Activation Functions

- **Preventing collapse:**
nonlinear function after each unit
- Learn nonlinear mappings from the input to the output
- Help the network to capture intricate structures in the data
- Act as a **gate** for information flow through the network



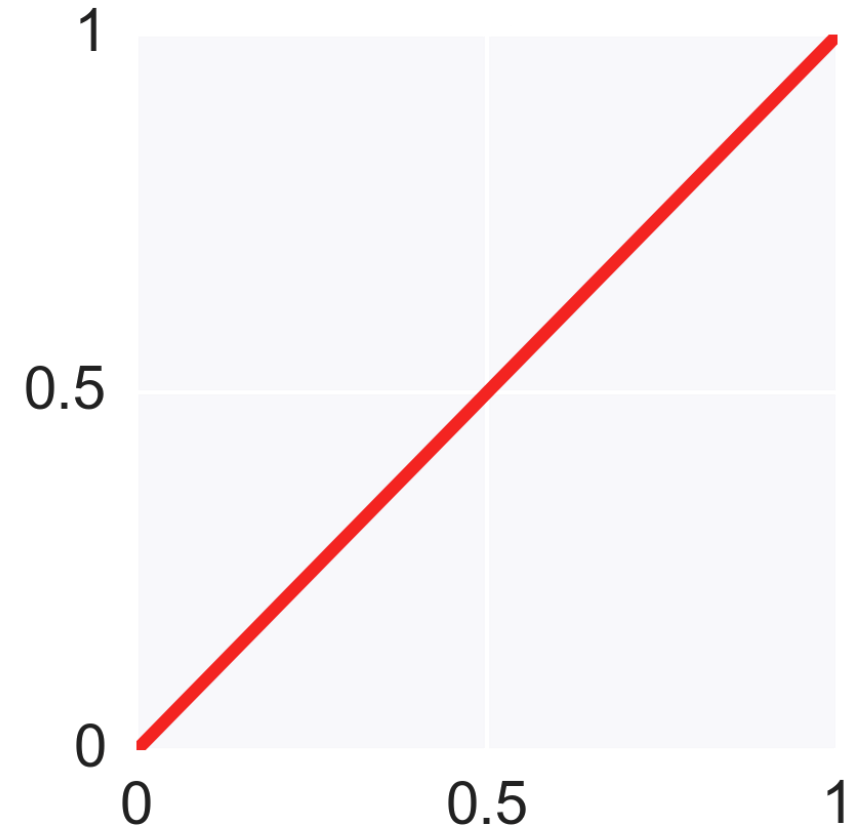
The Role of Activation Functions

The End

Linear Activation Functions

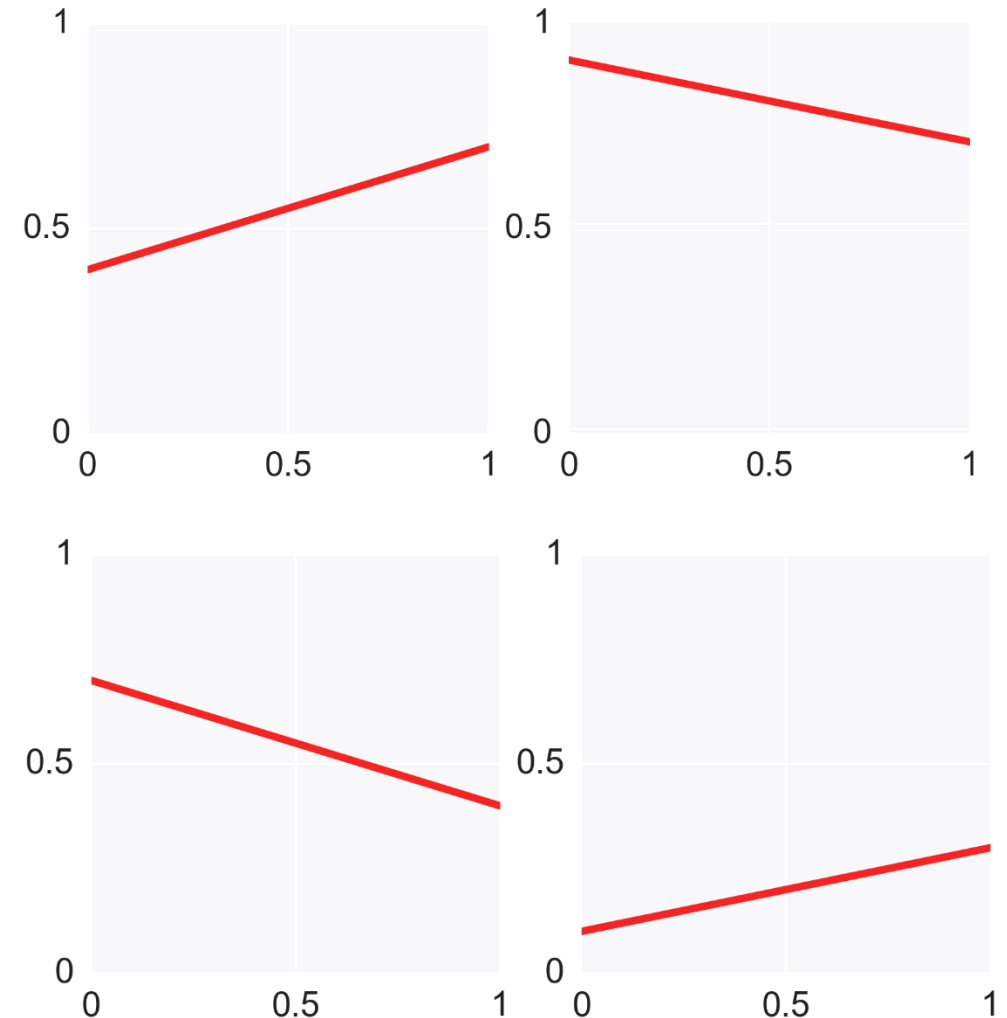
The Identity Function

- Identity function:
$$f(x) = x$$
- The output of this function is always **the same as the input** value.
- The input pass through the NN unit without any modification.



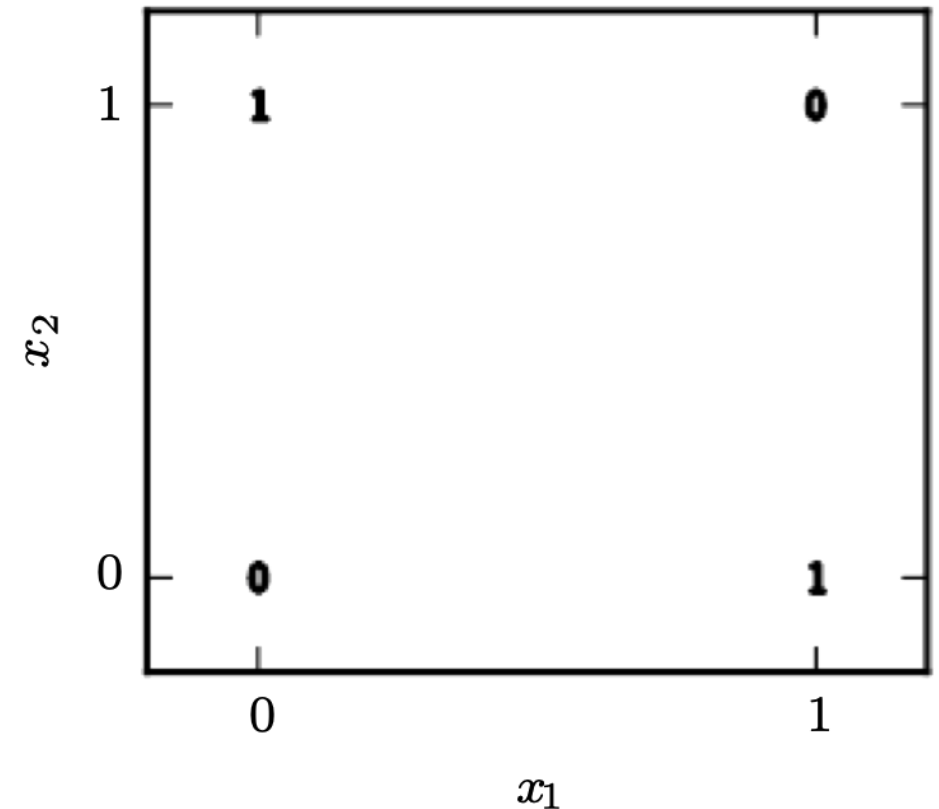
Other Variants

- Linear function variants (different linear equations)
- Mathematically, they are only doing multiplication and addition to the input
- Linear combinations of the input suffer from **network collapse**



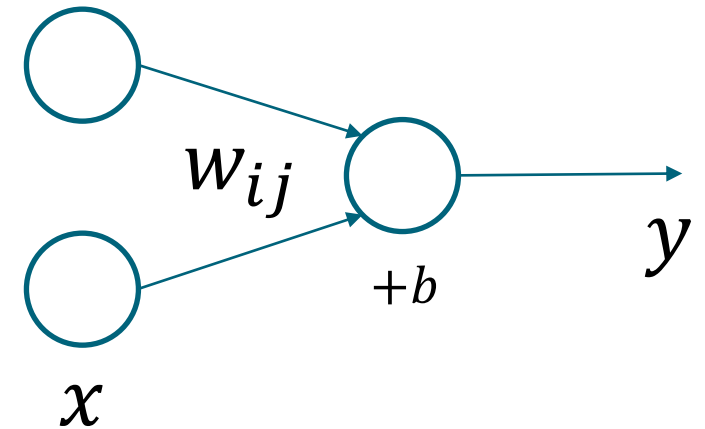
Example: Learning XOR, Part I

- Some simple functions cannot be learned with linear activations (i.e., exclusive)
- The XOR function is a binary operation with two inputs: x_1 and x_2
- The output is true (1) if inputs are different and false (0) otherwise



Example: Learning XOR, Part II

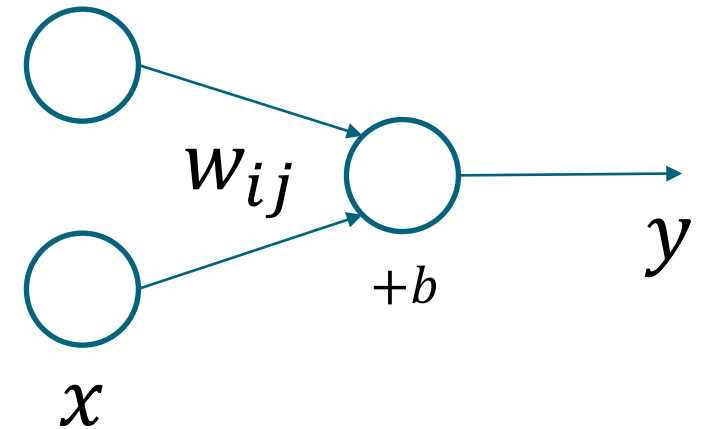
- The XOR function is a binary operation with two inputs: x_1 and x_2
- The output is true (1) if inputs are different and false (0) otherwise



$$y = w_1x_1 + w_2x_2 + b$$

Example: Learning XOR, Part III

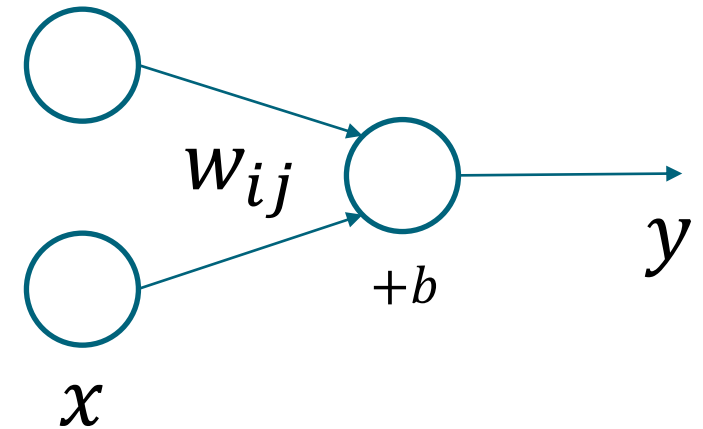
- The XOR function is a binary operation with two inputs: x_1 and x_2
- The output is true (1) if inputs are different and false (0) otherwise
- Learn weights for these mappings: $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$



$$y = w_1x_1 + w_2x_2 + b$$

Example: Learning XOR, Part IV

- The XOR function is a binary operation with two inputs: x_1 and x_2
- The output is true (1) if inputs are different and false (0) otherwise
- Learn weights for these mappings: $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$



$$0 = w_1 0 + w_2 0 + b$$

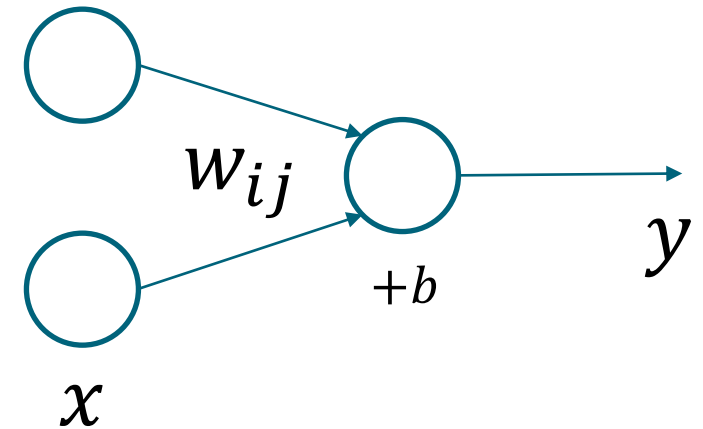
$$1 = w_1 0 + w_2 1 + b$$

$$1 = w_1 1 + w_2 0 + b$$

$$0 = w_1 1 + w_2 1 + b$$

Example: Learning XOR, Part V

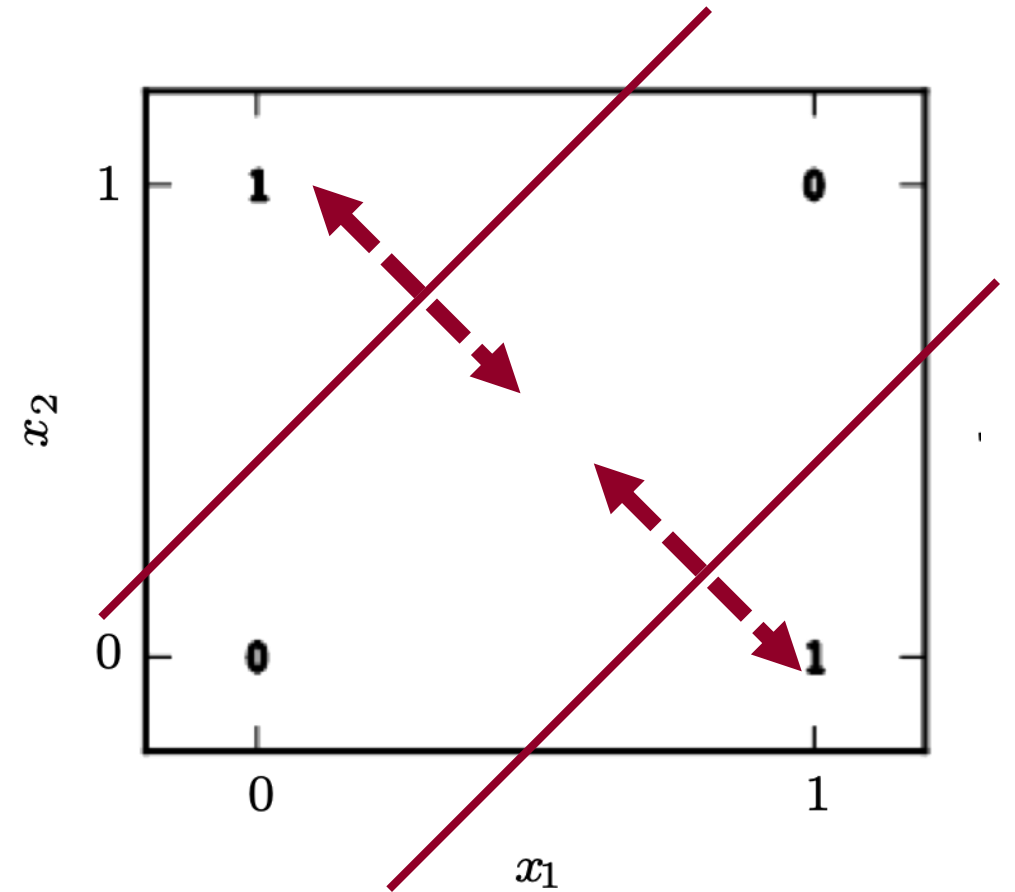
- The XOR function is a binary operation with two inputs: x_1 and x_2
- The output is true (1) if inputs are different and false (0) otherwise
- Learn weights for these mappings: $(0, 0) \rightarrow 0$, $(0, 1) \rightarrow 1$, $(1, 0) \rightarrow 1$, $(1, 1) \rightarrow 0$



$$\begin{aligned}0 &= b \\1 &= w_2 \\1 &= w_1 \\0 &= w_1 + w_2\end{aligned}$$

Example: Learning XOR, Part VI

- The proposed linear model can now describe the XOR function: is not linearly separable
- Motivates the need for other activation functions that can capture complex behavior



Pros and Cons of Linear Activation

- Advantages of linear activation functions
 - Simple and computationally efficient
 - Easy to interpret and understand
- Disadvantages of linear activation functions
 - Limited capacity for learning complex patterns
 - Stacked layers with linear activation functions still produce linear transformations (network collapse)
 - Rarely used anywhere but the output neurons

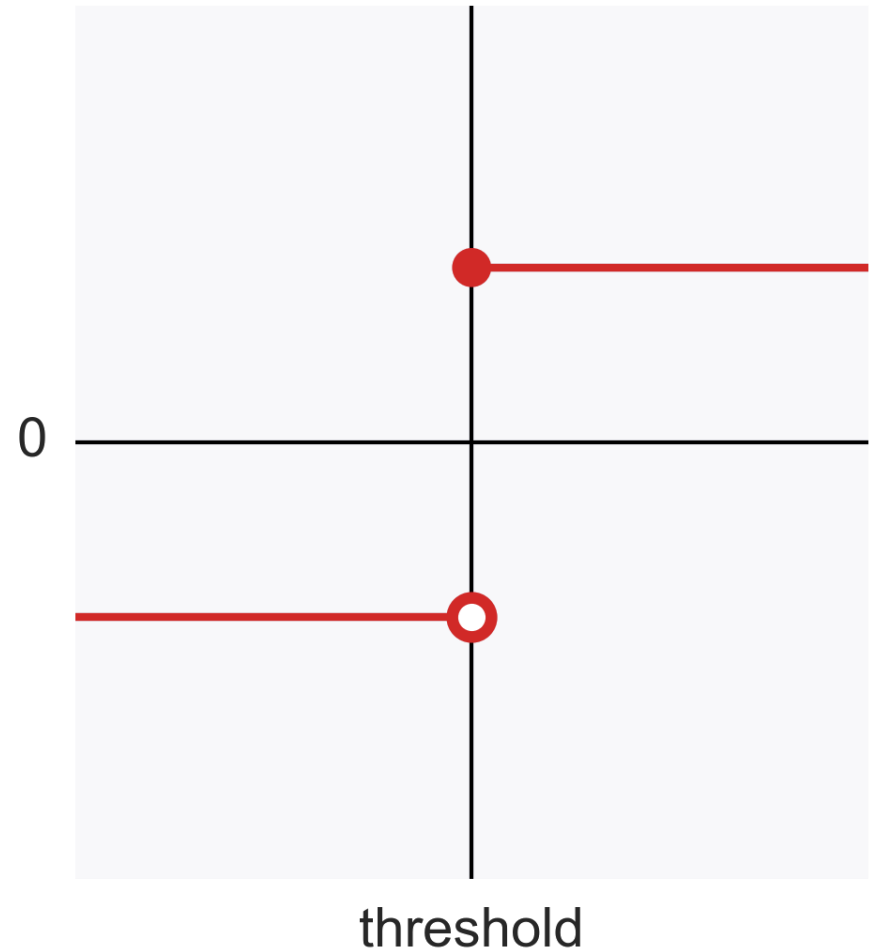
Linear Activation Functions

The End

Step Functions

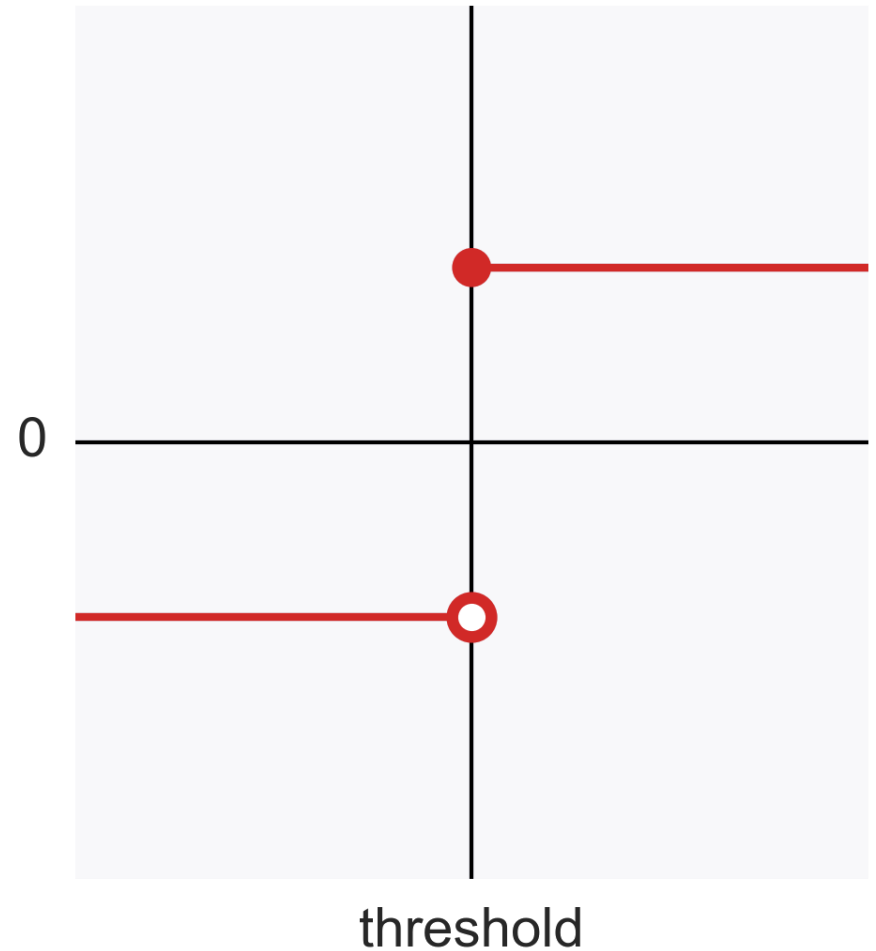
Step Functions, Part I

- The simplest activation function is the identity function, which is the same as having no activation function at all.
- The simplest nonlinear activation function is the **step function**.



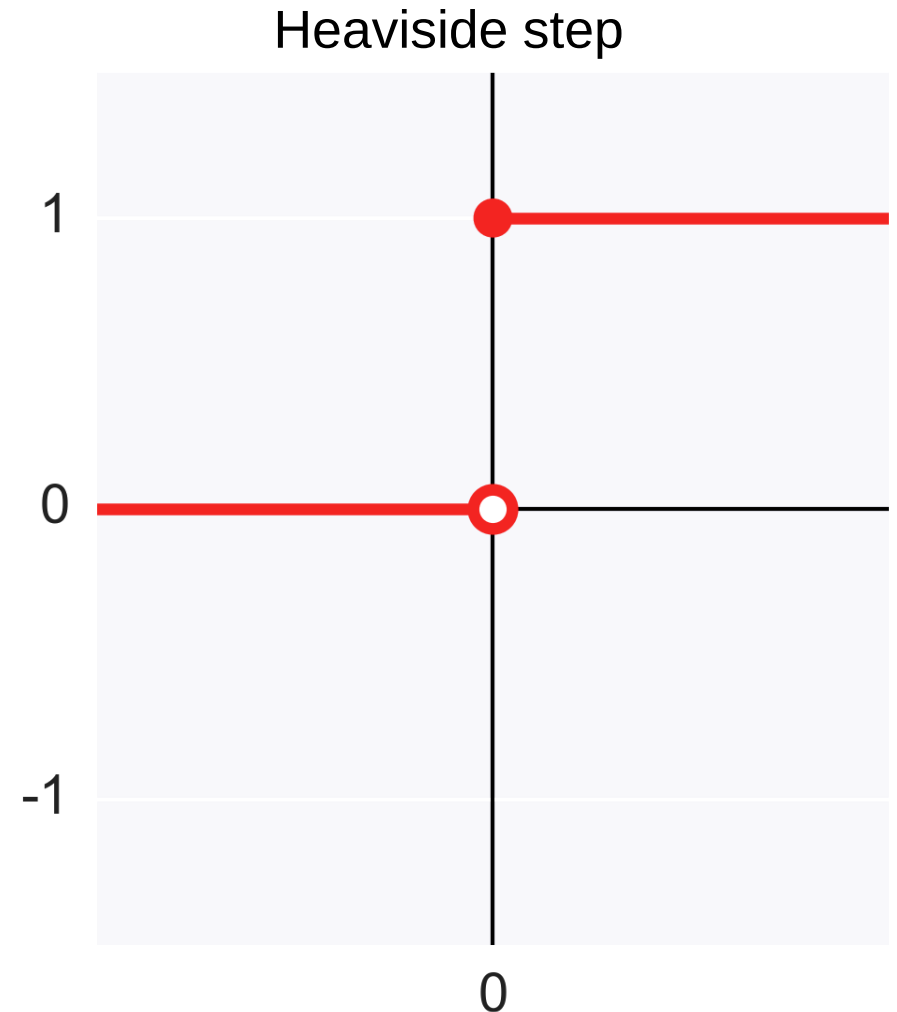
Step Functions, Part II

- A step function has **two fixed values**, one each to the left and right of a threshold value.



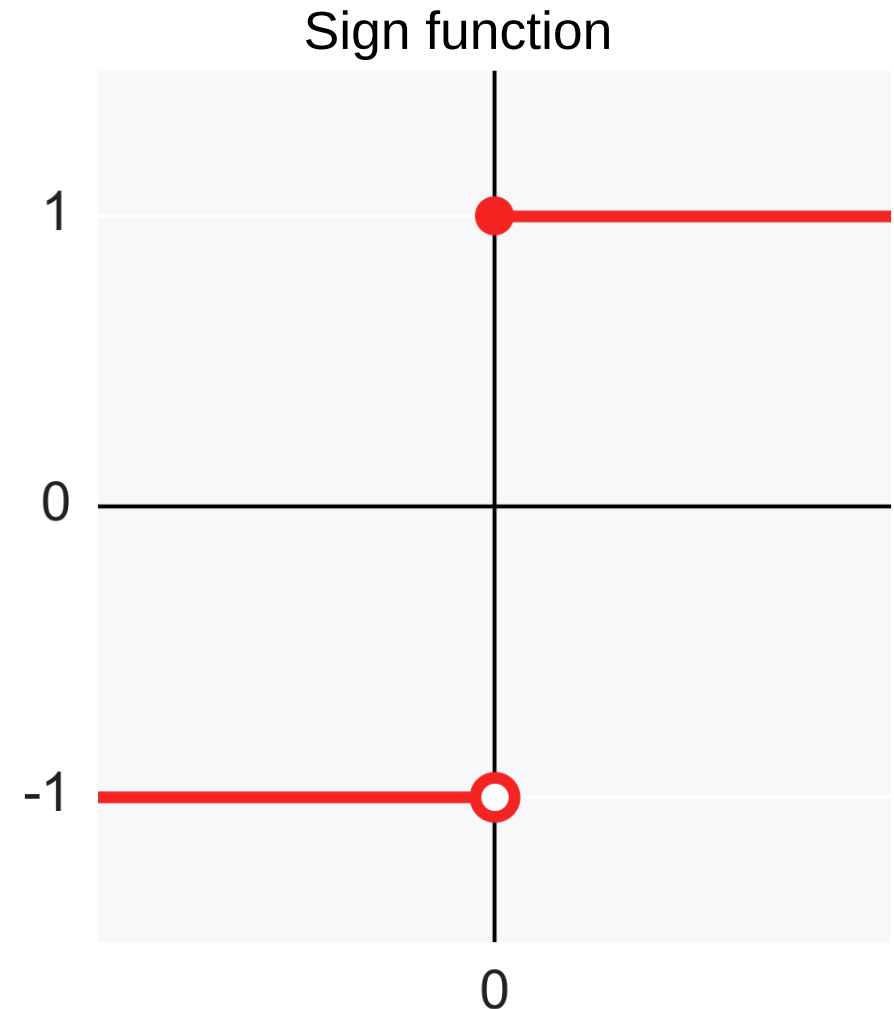
Step Functions, Part III

- A step function has **two fixed values**, one each to the left and right of a threshold value.
- If the threshold value is 0, then we give it the more specific name of the **Heaviside step**.



Step Functions, Part IV

- A step function has **two fixed values**, one each to the left and right of a threshold value.
- If the threshold value is 0, then we give it the more specific name of the **Heaviside step**.
- A Heaviside step function with the value -1 to the left is called a **sign function**.



Pros and Cons of Step Functions

- Advantages of step functions
 - Simplicity and easy implementation
 - Binary output suitable for certain tasks (e.g., binary classification)
 - Historically important (e.g., perceptron)
- Disadvantages of step functions
 - Non-differentiability at the threshold
 - Sensitivity to small changes in input
 - Lack of gradient information for learning

Step Functions

The End

Piecewise Linear Functions

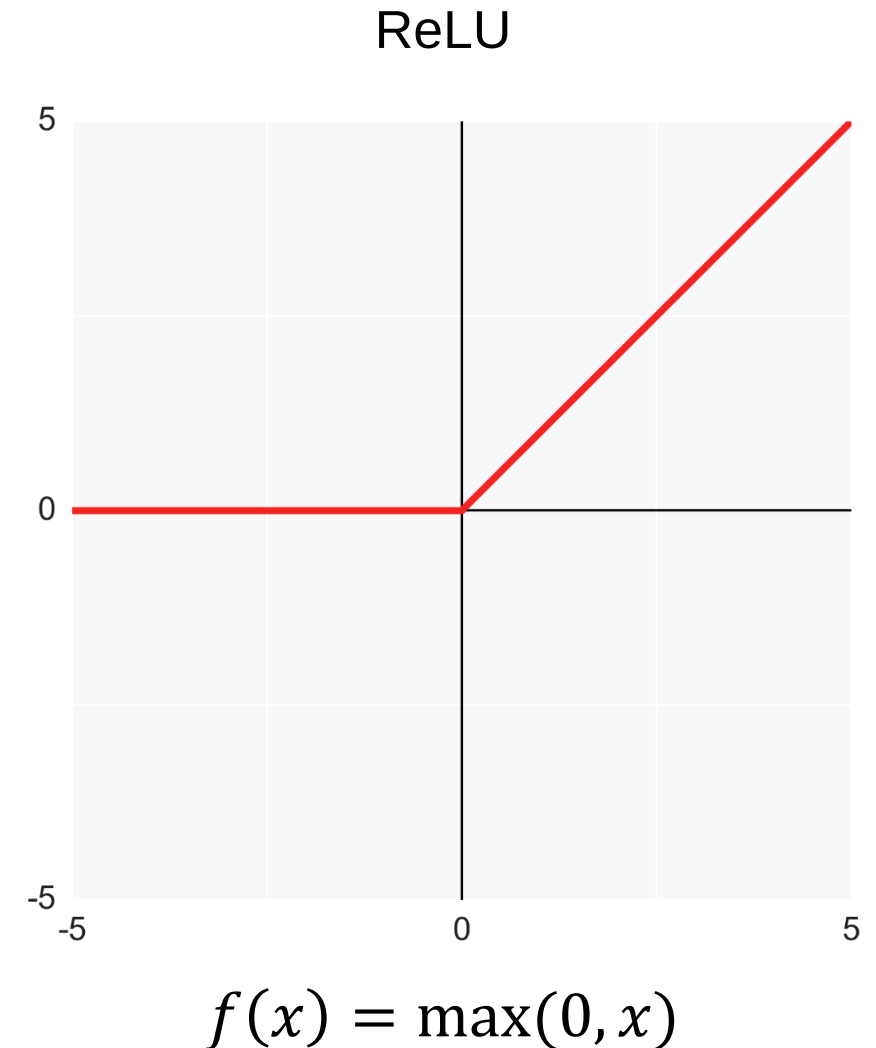
ReLu

Piecewise Linear Functions

- **Piecewise linear:** a function made up of several pieces, each of which is a straight line

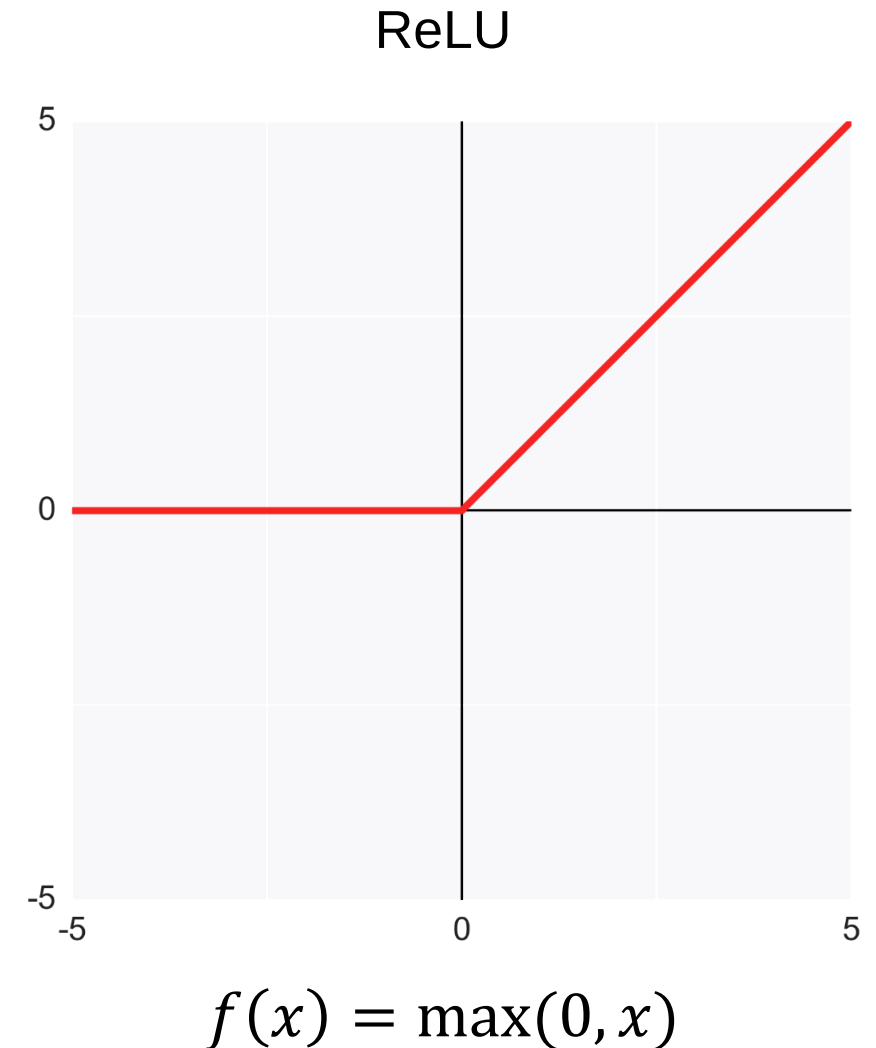
The Rectified Linear Unit (ReLU)

- **Piecewise linear:** a function made up of several pieces, each of which is a straight line
- Popular piecewise activation function: **rectified linear unit (ReLU)**



The Rectified Linear Unit (ReLU)

- **Piecewise linear:** a function made up of several pieces, each of which is a straight line
- Popular piecewise activation function: **rectified linear unit (ReLU)**
- If the input is less than 0, then the output is 0; otherwise, the output is the same as the input

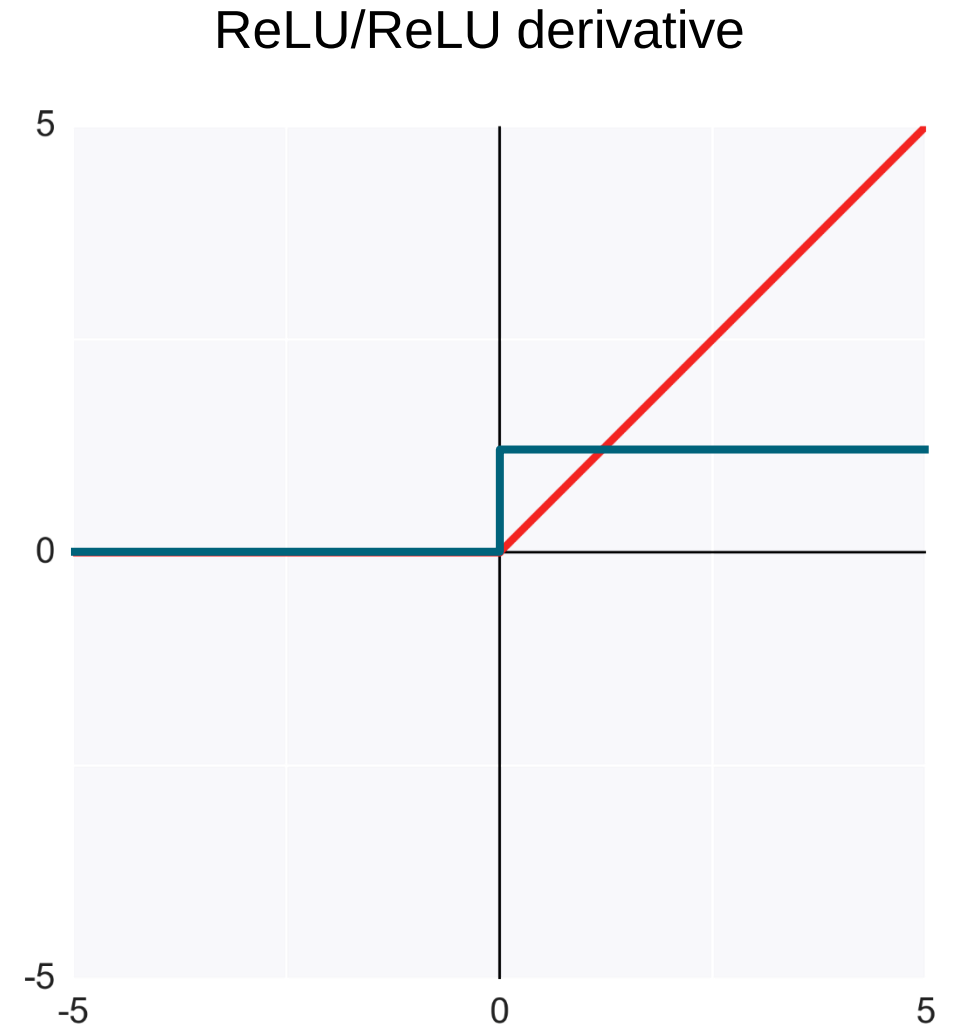


The Vanishing Gradient Problem

- **Vanishing gradient:** Gradients become very small as they are backpropagated through the layers.

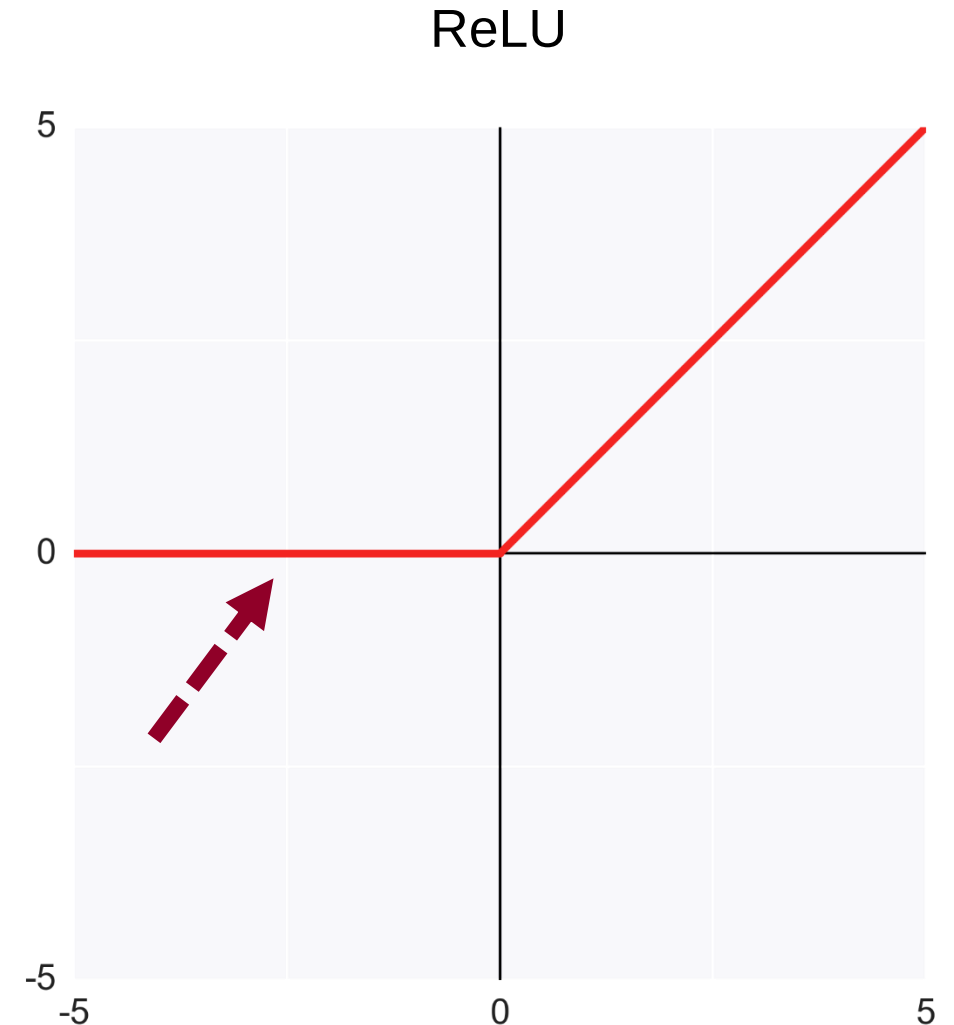
The Vanishing Gradient Problem

- **Vanishing gradient:** Gradients become very small as they are backpropagated through the layers.
- The ReLU derivative is either 0 or 1, depending on the input value, alleviating the vanishing gradient problem.



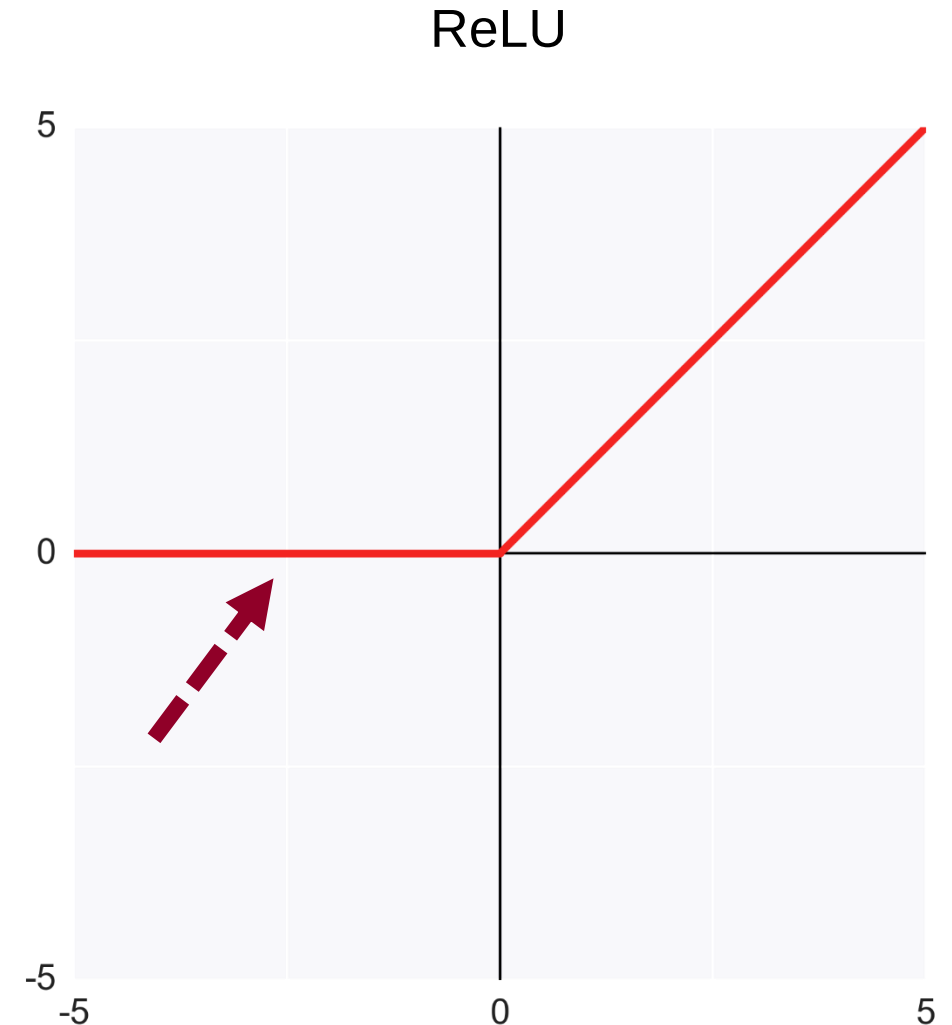
The Dying ReLU Problem

- The ReLU function can lead **to dead neurons**; some neurons in the network become always inactive.



The Dying ReLU Problem

- The ReLU function can lead **to dead neurons**; some neurons in the network become always inactive.
- During backpropagation, the gradient for these neurons will also be zero, **preventing any weight updates**.



ReLU Pros and Cons

- Advantages of ReLU activations
 - ReLU is a simple and computationally efficient function
 - Introduces nonlinearity into the network, enabling it to learn complex patterns
 - Helps address the vanishing gradient problem, which can occur in deep networks with traditional activation functions
- Disadvantages of ReLU activations
 - Neurons with negative input can become inactive, leading to dead neurons and reduced learning capacity

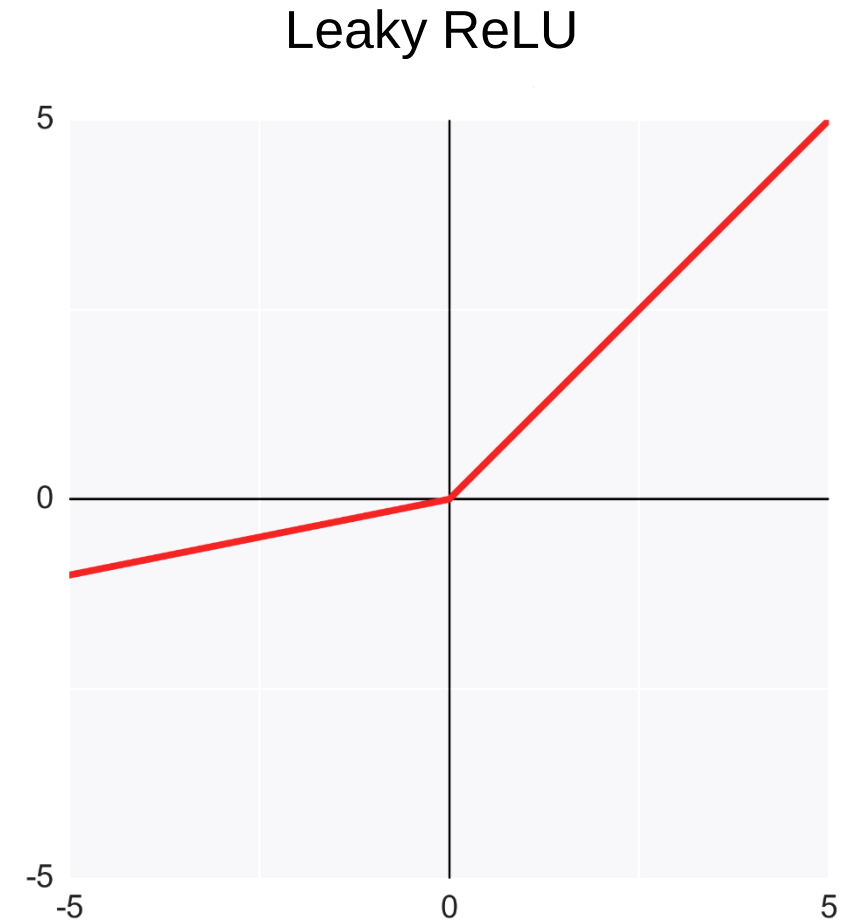
Piecewise Linear Functions

The End

Other Piecewise Linear Functions

The Leaky ReLU

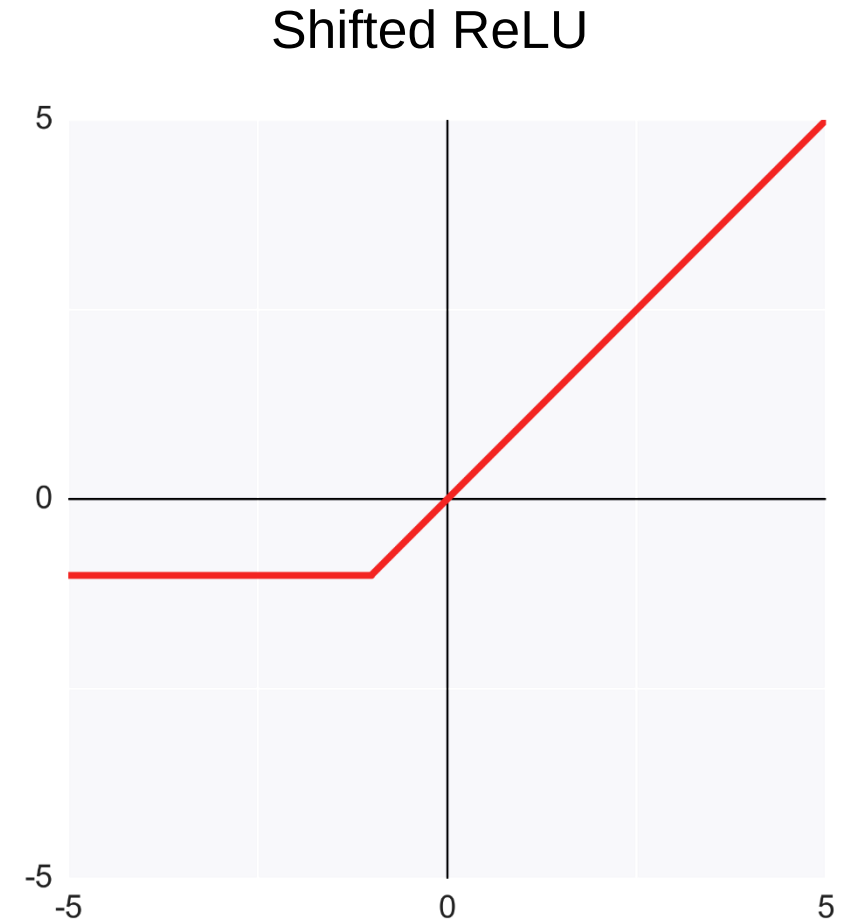
- The **leaky ReLU** changes the response for negative values.
- It outputs the input scaled down by a factor (e.g., 10).
- If factor equals 1, the function would be a straight line, and we risk that this neuron will collapse.



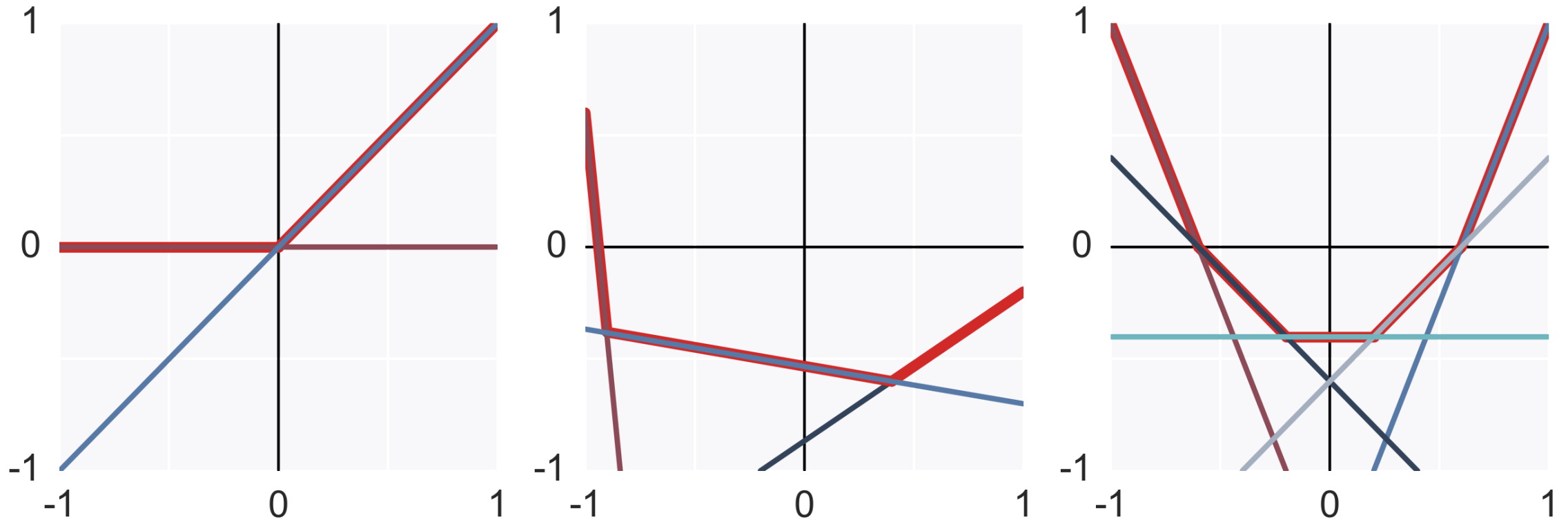
$$f(x) = \max(ax, x), a = 0.01$$

The Shifted ReLU

- The **shifted ReLU** moves the bend down and left.
- Allow the function to have a non-zero output for negative inputs.



Maxout Activation Functions



ReLU Variants: Pros and Cons

- Advantages of ReLU variants
 - Allows the activation threshold to be adjusted, which can be useful for specific problem domains
 - Avoids the dying ReLU problem
 - Maintains the benefits of ReLU, including nonlinearity and computational efficiency
- Disadvantages of ReLU variants
 - Introduces additional hyperparameters, like the shift parameter, that requires tuning

Other Piecewise Linear Functions

The End

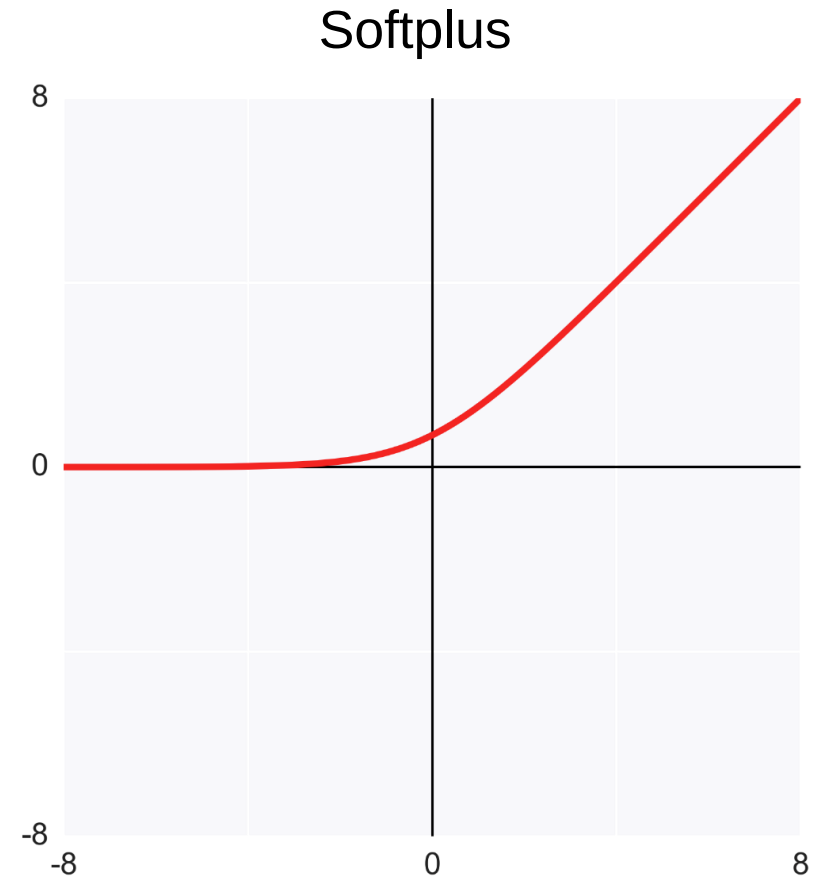
Smooth Functions

Smooth Functions

- Piecewise linear functions are easy to implement and computationally efficient
- Non-differentiable at the **inflection** point, which needs to be arbitrarily chosen
- Smooth activation functions are **continuous and fully differentiable** across their entire domain
- The continuous gradients can result in more stable optimization (in some cases)

The Softplus Function

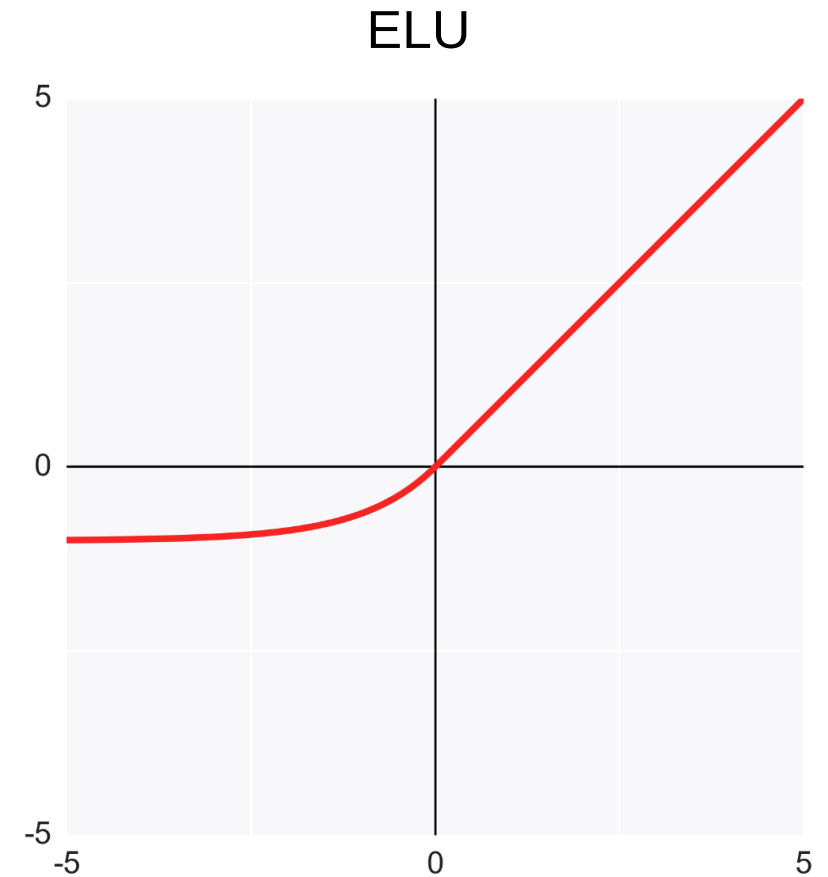
- Smooth version of the ReLU activation function
- Continuous and differentiable everywhere
- More **computationally expensive** than ReLU due to the exponential and logarithmic operations



$$f(x) = \ln(1 + e^x)$$

The Exponential Linear Unit (ELU)

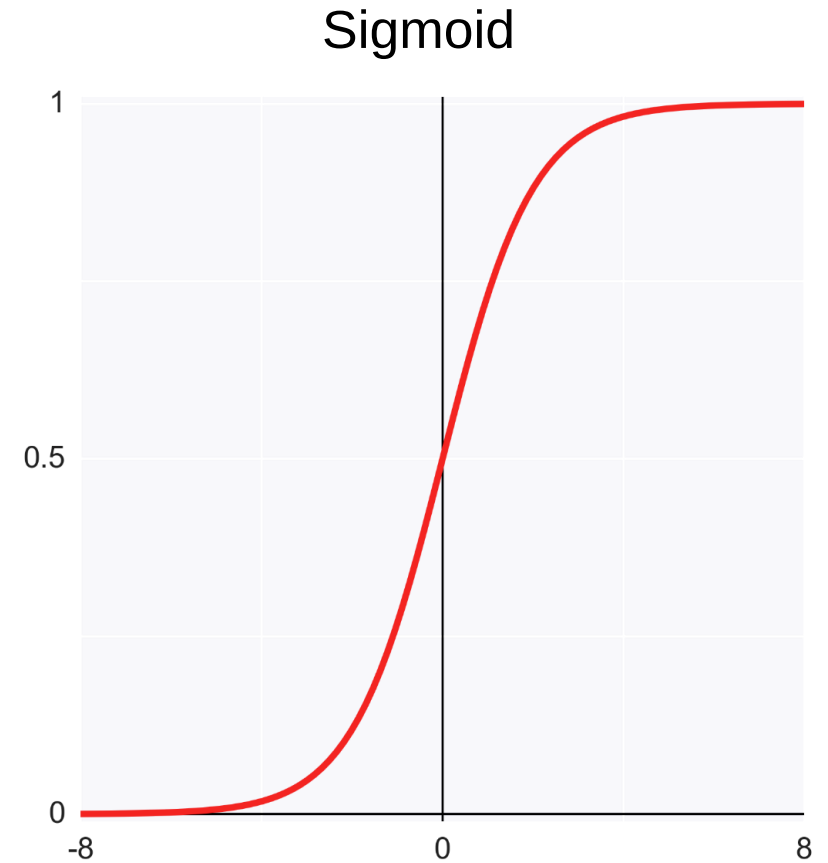
- Smooth version of the shifted ReLU activation function
- Allows a **small, non-zero gradient for negative input values**
- Parameter α is a positive scalar, typically set to 1



$$f(x) = \begin{cases} x & x \geq 0 \\ \alpha(e^x - 1) & x < 0 \end{cases}$$

The Sigmoid/Logistic Function

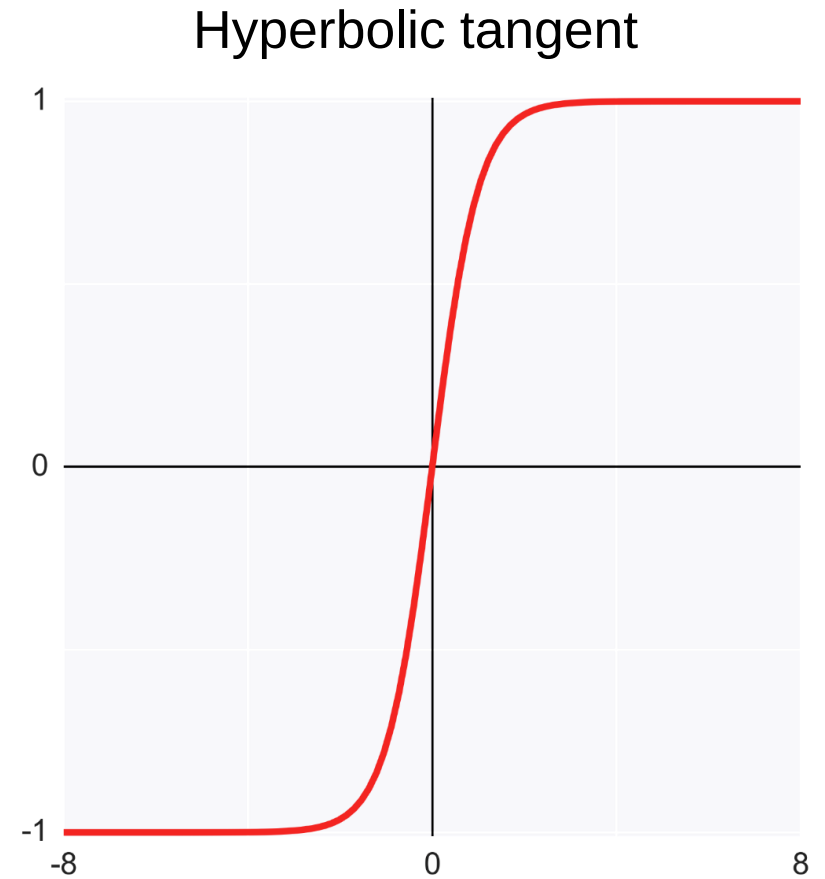
- S-shaped curve widely used in neural networks
- Its output range between 0 and 1 is useful for tasks such as **classification**
- Gradients become very small for input values far from 0: **vanishing gradient** problem



$$f(x) = \frac{e^x}{1 + e^x}$$

The Hyperbolic Tangent (Tanh)

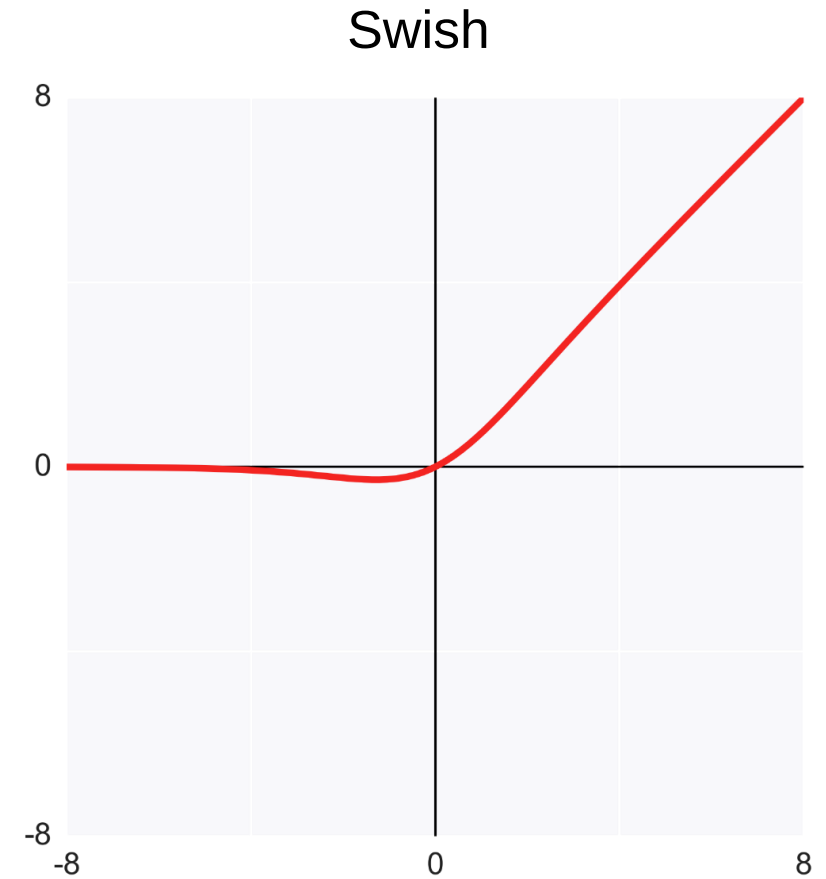
- Like the sigmoid function, the **hyperbolic tangent** exhibits a smooth, S-shaped curve
- The output range is from -1 to 1 , centered around 0
- Computationally expensive
- Suffers from the **vanishing gradient problem**



$$f(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}}$$

The Swish (or SiLU) Function

- Introduced by researchers at Google Brain
- Is a combination of the basic ReLU and sigmoid shapes
- The parameter β can be either a learnable parameter or a fixed scalar, typically set to 1
- **Self-gating mechanism**



$$f(x) = \frac{x}{1 + e^{-\beta x}}$$

Smooth Functions Pros and Cons

- Smoothness and differentiability
 - Suitable for various optimization algorithms
 - Better convergence properties in some cases
- Saturation and vanishing gradients
 - Sigmoid and tanh can suffer from vanishing gradients
 - Swish and ELU mitigate the issue to some extent

Smooth Functions Pros and Cons (cont.)

- Computational cost
 - Generally more expensive than piecewise linear functions
 - Exponential and multiplicative operations involved
- Specific advantages
 - Sigmoid: output range $(0, 1)$ useful for classification
 - tanh: centered output range $(-1, 1)$ for better convergence
 - Swish: self-gating mechanism for adaptability
 - ELU: addresses dying ReLU problem

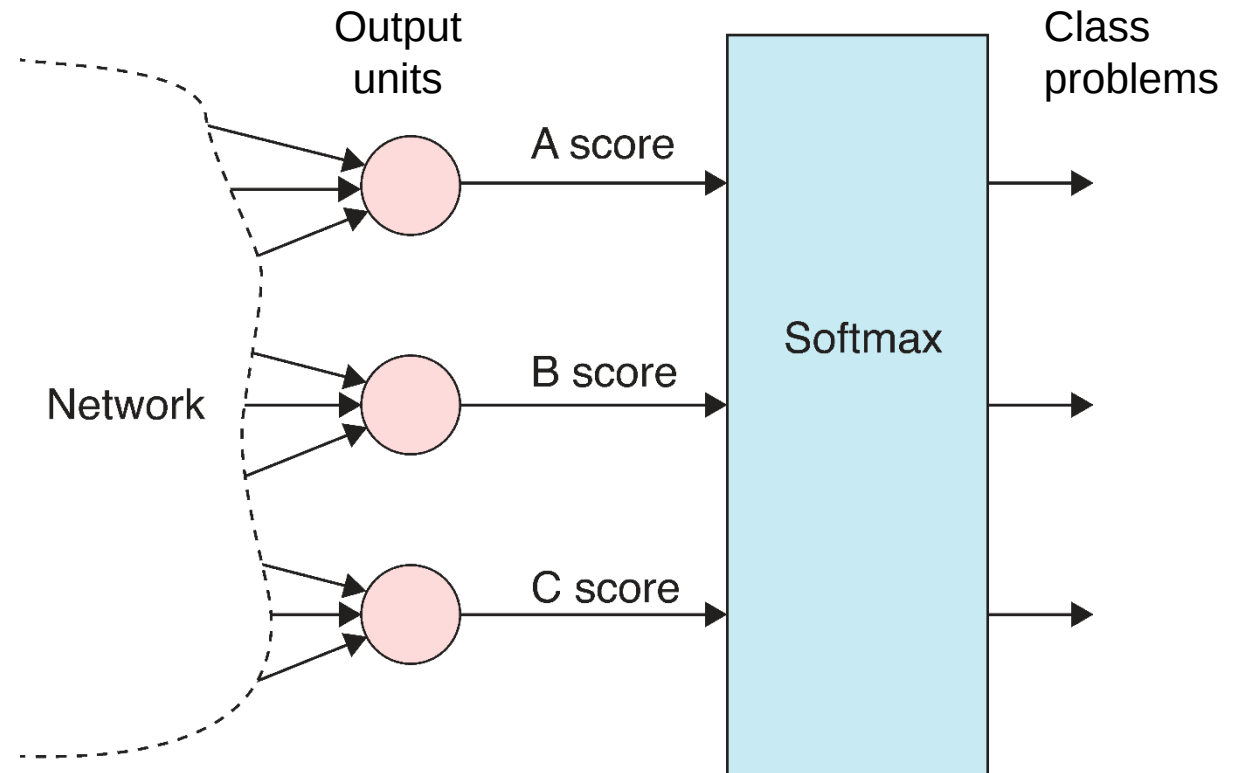
Smooth Functions

The End

The Softmax Function

The Softmax Function, Part I

- Special operation
- Typically applied **only at the output** neurons of a classifier
- Not strictly an activation function: it **applies simultaneously to all** the output neurons



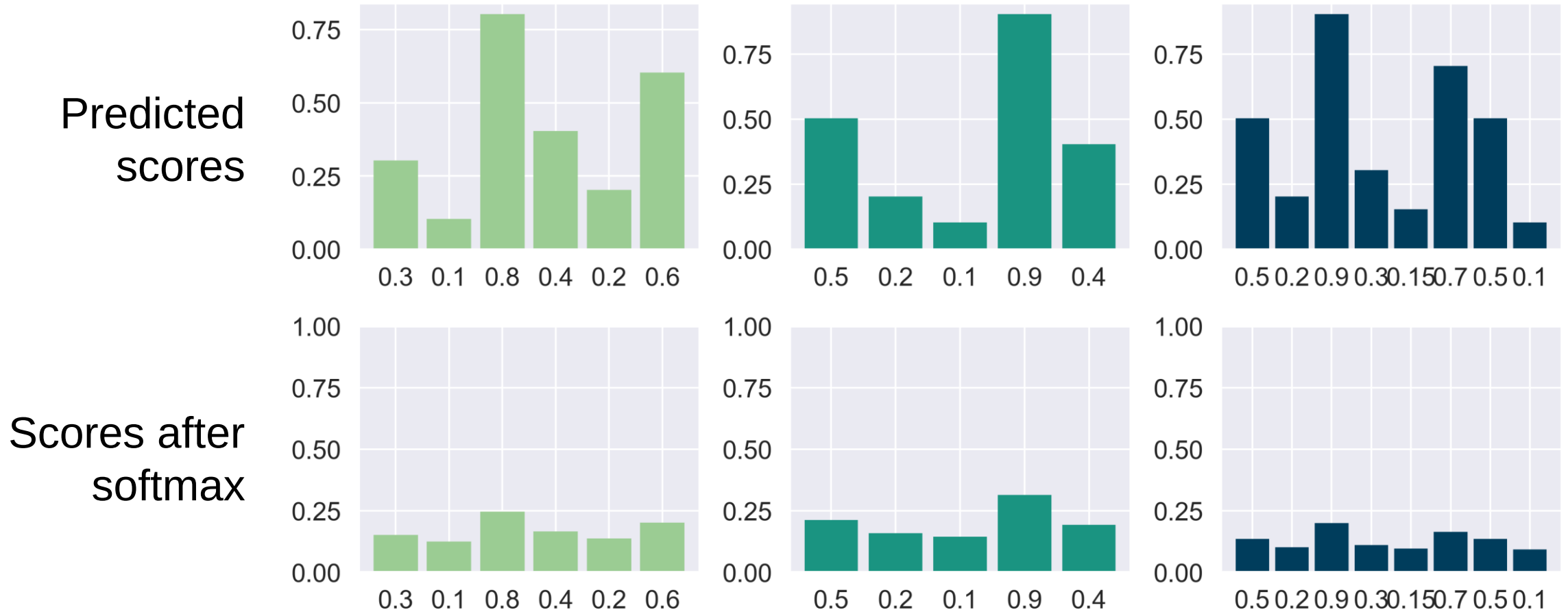
The Softmax Function, Part II

- Takes an input vector x of K real numbers and normalizes it into a probability distribution consisting of K probabilities

$$\text{Softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^K e^{x_j}}$$

- **Normalized exponential function** ensures that the sum of probabilities across all classes is equal to 1

The Softmax Function, Part III



The Softmax Function

The End

Choosing the Right Activation Function

Choosing the Right Activation Function

- Problem domain and characteristics
- Network depth and architecture
- Balance between complexity and performance
- Experimentation and validation

Choosing the Right Activation Function (cont.)

- Keep up with the latest research
- Custom activation functions
- Transfer learning and pretrained models
- Continuous learning and improvement

Choosing the Right Activation Function

The End